

Vue 3 + Pinia + LocalStorage

Vue 3 TODO 앱 완전 정복

실전 종합 실습: 기획부터 배포까지

Vue 3 Composition API

Pinia 상태관리

LocalStorage

Component Design

전체 강의 개요

1 기획 및 컴포넌트 설계

기능 정의와 컴포넌트 분해, 데이터 흐름 설계
#TodoApp #TodoInput #TodoList #TodoItem

2 TODO CRUD 구현

입력 / 목록 / 완료 / 삭제 기능 구현 + LocalStorage 저장 #CRUD
#LocalStorage #EventHandling

3 Pinia 상태관리 적용

상태 중앙관리, Getters / Actions 활용, 컴포넌트 단순화
#Pinia #Store #StateManagement

핵심 학습 포인트

Props / Emit 데이터 흐름

Computed / Watch 활용

Lifecycle (onMounted)

Pinia 상태관리 아키텍처

컴포넌트 구조 설계 능력

"종합 실습 프로젝트"

이론을 실제 코드로 연결하는 과정

01

CHAPTER

기획 및 컴포넌트 설계

기능 정의와 구조 설계

학습 목표

간단한 기능을 기준으로 컴포넌트 구조와 역할을 명확히 분리하고
데이터 흐름을 설계합니다.

KEY CONCEPTS

단방향 데이터 흐름

Emit (이벤트 상향)

UI vs 상태 분리

기획: TODO 앱 기능 정의

우리가 만들 애플리케이션의 핵심 기능 4가지



새 할 일 추가

입력창에 텍스트를 입력하고
엔터/버튼으로 목록에 추가



목록 보기

추가된 할 일들을
최신순/등록순으로 나열



완료 체크

체크박스를 통해
완료/미완료 상태 토글



항목 삭제

삭제 버튼을 클릭하여
목록에서 영구 제거

실습 설계 철학

상위 컴포넌트 (Container)

데이터(State)를 소유하고, 실제 로직(추가/삭제 등)을 처리합니다.

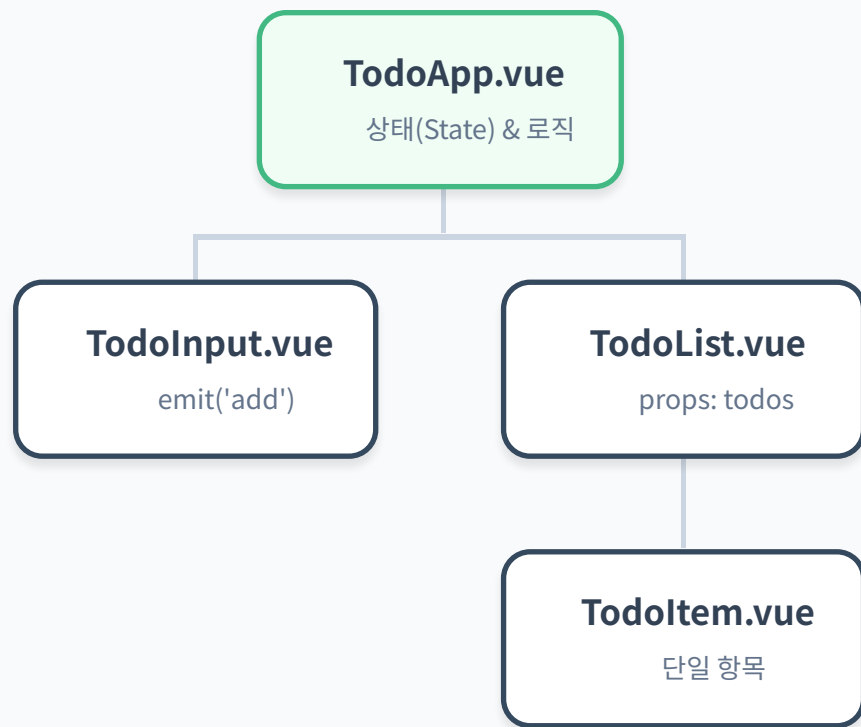
하위 컴포넌트 (UI)

props로 데이터를 받아 화면에 그리고,
이벤트(emit)로 사용자의 행동을 상위에 알리기만 합니다.

"단방향 데이터 흐름"

Props Down, Events Up

컴포넌트 구조 설계



데이터의 소유권 (Owner)

모든 데이터(todos)는 최상위 TodoApp이 관리합니다.
하위 컴포넌트는 데이터를 직접 수정하지 않습니다.

단방향 데이터 흐름

Props Down: 데이터는 위에서 아래로
Emit Up: 이벤트는 아래에서 위로 전달되는
Vue의 기본 패턴을 엄격히 준수합니다.

역할의 분리 (SoC)

상위: 데이터 저장 및 비즈니스 로직 처리
하위: UI 렌더링 및 사용자 입력 감지

02

CHAPTER

TODO CRUD 구현

핵심 기능 구현과 데이터 영속화

학습 목표

입력, 목록, 완료, 삭제의 CRUD 전 과정을 구현하고,
LocalStorage를 연동하여 데이터 저장 기능을 완성합니다.

KEY CONCEPTS

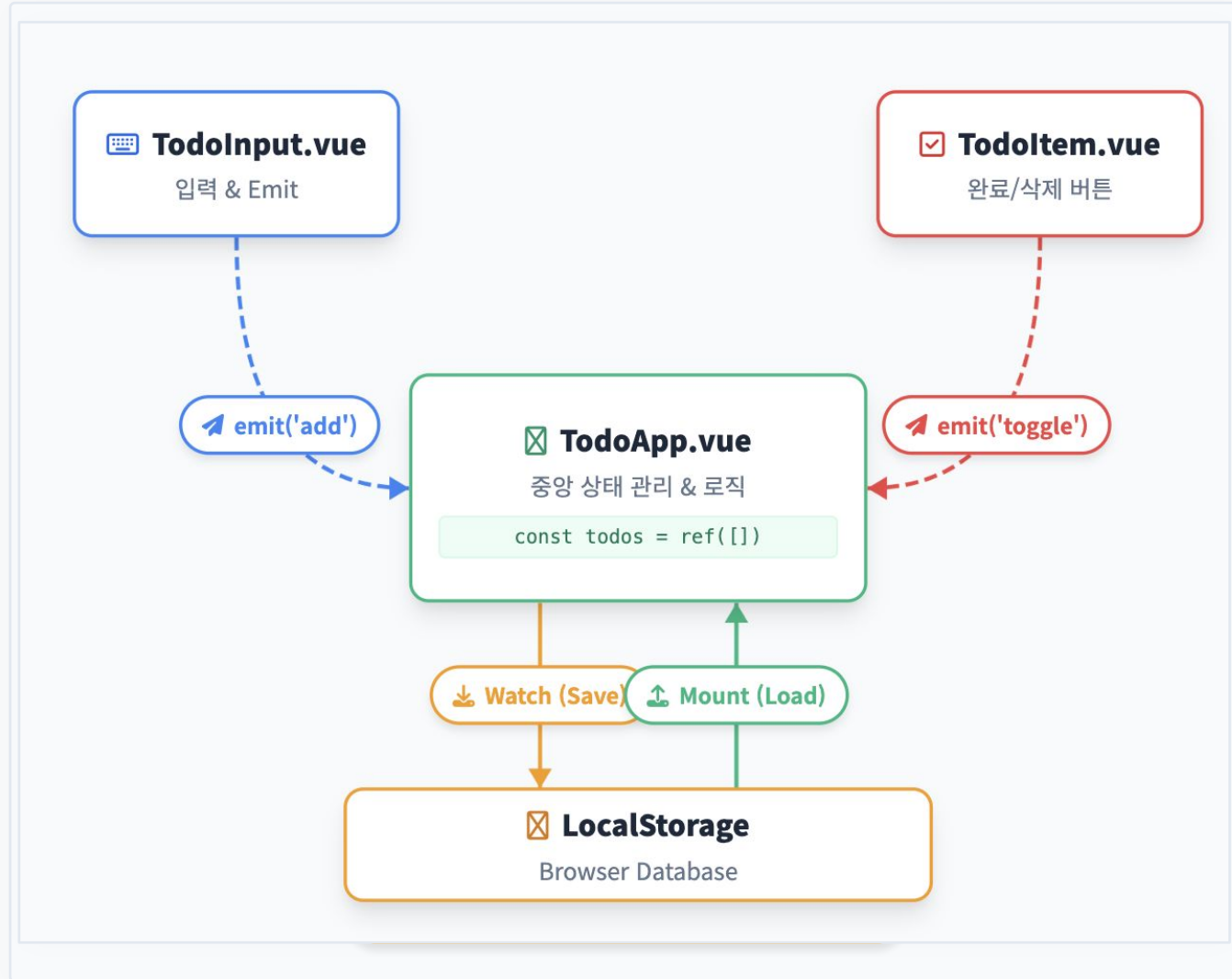
CRUD 로직

LocalStorage

Watch (자동저장)

v-model

구현 순서 및 데이터 흐름



1 입력 및 이벤트 발신

TodoInput에서 입력 확인 후 `emit('add-todo')`로 텍스트 전달

2 데이터 추가 (State Update)

TodoApp에서 이벤트를 받아 `todos.push()`로 배열에 객체 추가

3 항목 조작 요청

TodoItem에서 버튼 클릭 시 `emit('delete')` 또는 `emit('toggle')` 발신

4 상태 반영

TodoApp이 ID를 기준으로 해당 항목을 찾아 수정하거나 필터링하여 삭제

5 데이터 영속화

`onMounted`로 불러오고,
`watch`로 변경 사항 자동 저장 (Sync)

TodoInput.vue

사용자 입력을 받아 상위로 전달하는 컴포넌트

src/components/ToDoInput.vue

```
<template>
  <div class="input-box">
    <input
      v-model="text"
      @keyup.enter="add"
      placeholder="할 일을 입력하세요"
    />
    <button @click="add">추가</button>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const emit = defineEmits(['add-todo'])
const text = ref('')

function add() {
  if (text.value.trim() === '') return
  emit('add-todo', text.value)
  text.value = ''
}
</script>

<style scoped>
/* 스타일 생략 (flex 레이아웃 사용) */
</style>
```

양방향 바인딩 (v-model)

v-model="text"를 통해 입력값과 반응형 변수 text를 실시간으로 동기화합니다.

이벤트 전달 (Emit)

입력된 값은 직접 처리하지 않고, emit('add-todo', 값)을 통해 상위 컴포넌트(TodoApp)로 올려보냅니다.

유효성 검사

빈 문자열이 입력되지 않도록 trim() 체크 후 전송하며, 전송 후에는 입력창을 초기화합니다.

TodoItem.vue

개별 할 일 항목을 표시하고 상위로 이벤트를 전달하는 컴포넌트

src/components/ToDoItem.vue

```
<template>
  <div class="item">
    <input type="checkbox" v-model="localDone" @change="toggle" />
    <span :class="{ done: localDone }">{{ todo.text }}</span>
    <button @click="remove">삭제</button>
  </div>
</template>

<script setup>
import { ref } from 'vue'

const props = defineProps({
  todo: Object
})

const emit = defineEmits(['toggle-done', 'delete-todo'])

// Props는 읽기 전용이므로 로컬 상태로 복사
const localDone = ref(props.todo.done)

function toggle() {
  emit('toggle-done', props.todo.id)
}

function remove() {
  emit('delete-todo', props.todo.id)
}
</script>

<style scoped>
.done { text-decoration: line-through; color: gray; }
</style>
```

Props 수신

부모 컴포넌트로부터 todo 객체(데이터)를 defineProps를 통해 전달받아 렌더링에 사용합니다.

단방향 데이터 흐름

Vue에서 props는 읽기 전용입니다. 체크박스 바인딩을 위해 localDone이라는 별도의 로컬 상태를 만들어야 에러를 방지할 수 있습니다.

이벤트 전송 (Emit)

삭제나 완료 상태 변경 시 직접 데이터를 수정하지 않고, emit을 통해 '나 삭제됐어', '나 토글됐어'라고 부모에게 신호만 보냅니다.

TodoList.vue

할 일 목록을 관리하고 개별 항목을 반복 렌더링하는 래퍼 컴포넌트

src/components/ToDoList.vue

```
<template>
  <div class="list">
    <ToDoItem
      v-for="item in todos"
      :key="item.id"
      :todo="item"
      @toggle-done="toggle"
      @delete-todo="remove"
    />
  </div>
</template>

<script setup>
import ToDoItem from './ToDoItem.vue'

const props = defineProps({
  todos: Array
})

const emit = defineEmits(['toggle-done', 'delete-todo'])

function toggle(id) {
  emit('toggle-done', id)
}

function remove(id) {
  emit('delete-todo', id)
}
</script>

<style scoped>
/* 스타일 생략 (flex-direction: column) */
</style>
```

리스트 렌더링 (v-for)

v-for="item in todos"를 사용하여 배열의 각 항목을 순회하며 컴포넌트를 생성합니다. 성능 최적화를 위해 :key 속성은 필수입니다.

Props 내려주기

상위(TodoApp)에서 받은 todos 배열 데이터를 :todo="item"을 통해 각각의 자식 컴포넌트 (ToDoItem)에게 전달합니다.

이벤트 중계 (Bubbling)

하위 컴포넌트에서 발생한 완료/삭제 이벤트를 직접 처리하지 않고, emit을 통해 다시 상위 컴포넌트 (TodoApp)로 전달하는 중간 다리 역할을 합니다.

TodoApp.vue

모든 데이터를 관리하고 로직을 수행하는 메인 컴포넌트

src/views/ToDoApp.vue

```
<template>
  <main class="page">
    <h1>TODO 리스트</h1>
    <!-- emit 이벤트 연결 -->
    <ToDoInput @add-todo="addTodo" />
    <ToDoList
      :todos="todos"
      @toggle-done="toggleDone"
      @delete-todo="deleteTodo"
    />
  </main>
</template>

<script setup>
import { ref, onMounted, watch } from 'vue'
import ToDoInput from '../components/ToDoInput.vue'
import ToDoList from '../components/ToDoList.vue'

const todos = ref([])

// 로컬 스토리지 저장/로드
function saveLocal() {
  localStorage.setItem('todos', JSON.stringify(todos.value))
}
function loadLocal() {
  const data = localStorage.getItem('todos')
  if (data) todos.value = JSON.parse(data)
}

onMounted(() => loadLocal())
watch(todos, () => saveLocal(), { deep: true })

function addTodo(text) {
  todos.value.push({ id: Date.now(), text, done: false })
}
function toggleDone(id) {
  const t = todos.value.find(v => v.id === id)
  if (t) t.done = !t.done
}
function deleteTodo(id) {
  todos.value = todos.value.filter(v => v.id !== id)
}
</script>
```

LocalStorage 영속화

onMounted 시점에 저장된 데이터를 불러오고, watch(deep: true)를 사용해 데이터 변경 시 자동으로 저장합니다.

Props & Emit 연결

:todos로 데이터를 내려주고, @add-todo 등의 이벤트 리스너로 하위 컴포넌트의 요청을 받아 처리합니다.

중앙 상태 관리

모든 데이터(State)는 최상위 컴포넌트인 TodoApp이 소유하며, CRUD 로직 또한 이곳에 집중되어 있습니다.

LocalStorage 저장/불러오기

데이터 영속화(Persistence)를 위한 핵심 로직 구현

src/views/ToDoApp.vue (Logic)

LocalStorage Logic

```
function saveLocal() {  
  // 배열을 문자열로 변환하여 저장  
  localStorage.setItem('todos', JSON.stringify(todos.value))  
}  
  
function loadLocal() {  
  const data = localStorage.getItem('todos')  
  // 데이터가 있을 때만 파싱  
  if (data) todos.value = JSON.parse(data)  
}  
  
// 1. 마운트 시 불러오기  
onMounted(() => {  
  loadLocal()  
})  
  
// 2. 변경 감지 시 자동 저장  
watch(todos, () => {  
  saveLocal()  
}, { deep: true })
```

onMounted (불러오기)

컴포넌트가 DOM에 부착된 직후(onMounted), 저장된 데이터를 한 번만 불러와 초기화합니다.

watch + deep (자동 저장)

{ deep: true } 옵션으로 객체/배열 내부의 값 변경까지 감지하여, 변경될 때마다 자동으로 LocalStorage에 덮어씁니다.

JSON 직렬화 필수

LocalStorage는 문자열만 저장할 수 있습니다. JSON.stringify와 JSON.parse를 사용해 배열 형태를 유지해야 합니다.

상태 중앙관리와 Pinia 적용

03

CHAPTER

학습 목표

상태(State)를 Store로 이동해 컴포넌트의 로직을 단순화하고 앱의 재사용성과 유지보수성을 높입니다.

KEY CONCEPTS

State / Getters / Actions

단일 진실 공급원 (SSOT)

프레젠테이션 컴포넌트

Pinia 적용 이유와 구조 변화

기존 방식의 한계 (Prop Drilling)

컴포넌트 규모가 커질수록 상위 컴포넌트(App.vue)에 모든 상태와 로직이 집중되고, 하위 컴포넌트로 데이터를 전달하는 과정이 복잡해집니다.

해결책: Store 도입 (SSOT)

데이터(State)와 로직(Actions)을 Store로 이동시켜 중앙에서 관리합니다. 컴포넌트는 단순히 화면을 그리고 이벤트를 호출하는 역할만 수행합니다.

구조 변화

BEFORE

TodoApp.vue

상태 + 로직 소유

하위 컴포넌트
(단순 Props/Emit)

AFTER (PINIA)

Pinia Store

상태 + 로직 중앙 관리

모든 컴포넌트
(Store 직접 접근)

Pinia 적용 포인트

State (상태) todos 배열을 Store의 state로 정의하여 어디서든 접근 가능하게 함

Actions (로직) add, toggle, remove 등 비즈니스 로직을 함수로 정의

Getters (계산된 값) completedCount와 같이 state를 가공한 값을 실시간으로 제공

Persistence (영속화) Action 내부에서 데이터 변경 후 save()를 호출하여 LocalStorage 동기화

💡 컴포넌트 코드가 50% 이상 줄어듭니다!

todoStore.js

Pinia를 이용한 전역 상태 관리 및 비즈니스 로직 분리

src/stores/todoStore.js

```
import { defineStore } from 'pinia'

export const useTodoStore = defineStore('todo', {
  // State: 반응형 데이터 (todos 배열)
  state: () => ({
    todos: []
  }),

  // Getters: state를 가공한 값 (computed와 유사)
  getters: {
    completedCount(state) {
      return state.todos.filter(t => t.done).length
    }
  },

  // Actions: 메서드 (비동기 로직, 상태 변경)
  actions: {
    load() {
      const data = localStorage.getItem('todos')
      if (data) this.todos = JSON.parse(data)
    },
    save() {
      localStorage.setItem('todos', JSON.stringify(this.todos))
    },
    add(text) {
      this.todos.push({
        id: Date.now(), text, done: false
      })
      this.save() // 변경 시 자동 저장 호출
    }
    // toggle, remove 생략...
  }
})
```

State & Getters

state는 데이터를, getters는 computed처럼 데이터를 가공한 결과(예: 완료된 개수)를 제공합니다.

Actions (로직의 중심)

컴포넌트에 흩어져 있던 add, save 등의 로직을 Store의 actions로 옮겨와 중앙에서 관리합니다.

데이터 영속화

데이터를 수정하는 액션(add, toggle 등) 내부에서 직접 save()를 호출하여, 항상 최신 상태를 LocalStorage에 동기화합니다.

TodoApp.vue (Pinia 버전)

Store와 연결하여 상태와 로직을 위임한 메인 컴포넌트

src/views/ToDoApp.vue

```
<template>
  <main class="page">
    <h1>TODO 리스트 (Pinia 버전)</h1>
    <!-- Getter 활용: 완료된 개수 표시 -->
    <p>완료된 항목: {{ completedCount }}개</p>

    <!-- Store의 액션을 바로 연결 -->
    <ToDoInput @add-todo="add" />
    <ToDoList
      :todos="todos"
      @toggle-done="toggle"
      @delete-todo="remove"
    />
  </main>
</template>

<script setup>
import { storeToRefs } from 'pinia'
import { onMounted } from 'vue'
import { useToDoStore } from '../stores/todoStore'
// ... 하위 컴포넌트 import 생략

const todoStore = useToDoStore()

// 1. 반응형 상태(State/Getters)는 storeToRefs로 추출
const { todos, completedCount } = storeToRefs(todoStore)

// 2. 함수(Actions)는 그냥 구조 분해 할당 가능
const { add, toggle, remove, load } = todoStore

onMounted(() => {
  load() // Store의 load 액션 호출
})
</script>
```

storeToRefs의 중요성

Store의 state나 getters를 그냥 구조 분해하면 반응성이 깨집니다. 반드시 storeToRefs를 사용해야 반응형 참조(Ref)로 유지됩니다.

간결해진 코드

이전의 saveLocal, addTodo 등의 복잡한 함수 구현이 모두 사라졌습니다. 컴포넌트는 이제 Store의 기능을 가져다 쓰기만 하면 됩니다.

이벤트 직접 연결

Store의 Action인 add, toggle 등을 템플릿 이벤트 핸들러로 직접 전달할 수 있어 코드가 매우 직관적입니다.

오늘 배운 핵심 내용



컴포넌트 구조 설계 & 데이터 흐름

Props Down, Emit Up 패턴 완벽 이해



CRUD 구현 & LocalStorage

생명주기(onMounted)와 Watch 활용



Pinia 상태관리 아키텍처

Store 분리 및 Actions/Getters 활용



수고하셨습니다!

이제 여러분은 Vue 3의 핵심 패턴을 실전 프로젝트에 적용할 수 있는 기초를 다졌습니다.